

Firestore 스키마 설계

이 스키마의 목적은 좋은 스키마가 아니다. 사람이 엑셀로 하던 가공 단계를 없애는 것이다.
JaionX에 들어가는 데이터가 지금과 동일해지는 순간, 구 DID를 폐기한다.

갱신 2026-09-02 검증 TACTIC.xlsx 1,536행 · D-MST 20팀 로직 app/utils/didLogic.ts

00 목표 — 스키마는 목적이 아니라 수단이다

지금 이 흐름에서 가운데 사람 손을 없애는 것이 전부다.

현재 DID 입력 → MySQL → 사람이 추출·엑셀 가공 → CSV → JaionX admin 업로드

목표 DID 입력 → Firestore → 파이썬 자동 변환 → JaionX 갱신
→ JaionX 데이터가 지금과 동일해지는 순간, 구 DID 폐기

규칙 0 — 다른 모든 원칙보다 위에 있다

JaionX가 받는 모든 컬럼은 Firestore 데이터만으로 복원 가능해야 한다. 복원 불가능한 값이 하나라도 있으면 그 값은 반드시 저장한다.

"계산으로 대체한다"는 결정은 복원 가능성이 증명될 때만 유효하다. 출력 규격의 실체는 §09에 있다.

01 설계 원칙

레거시를 존중하되 그대로 베끼지 않는 기준. 이후 모든 결정이 여기서 나온다.

00

JaionX 출력을 전부 복원할 수 있어야 한다

위 규칙 0. 나머지 원칙은 이것을 만족하는 선에서만 적용된다.

01

접두사를 버린다

SQL은 네임스페이스가 평면이라 `gr_` `gi_` 가 필요했다. Firestore는 컬렉션이 이미 네임스페이스다.

02

이름은 신용어를 쓴다

TMP→TAP, TAP→DAP, CTP→DTP, CSP→DSP, CT*→DT*. 레거시 이름은 §09 매핑표에만 남긴다.

03

계산되는 값은 저장하지 않는다

1,200건 전체 재계산이 **0.16ms**다(§13 실측). 저장은 어긋날 자리만 만든다.

04

단, 확정 시점 스냅샷은 남긴다

리포트·JaionX 동기화는 "지금 값"이 아니라 "그때 값"을 요구한다.

05

같이 읽는 건 한 문서로

라인업 18명은 map 필드. 계속 늘어나는 레코드 1,200건은 서브컬렉션.

06

이력은 원장 + 파생 뷰

이적시장 = 추가 전용 계약 원장. 현재 스쿼드 = 그걸로 만든 조회용 뷰.

07

과거는 과거 모습대로

이적·등번호 변경이 작년 기록을 바꾸면 안 된다. 라인업에 이름·번호 스냅샷을 남긴다.

08

모든 값은 출처를 남긴다

사람 입력 / 비전 / 보정을 구분하지 않으면 나중에 신뢰도 판단이 불가능하다.

02 역할 분담 — 무엇을 어디서 계산하나

판정 로직의 권위는 파이썬에 있다. 클라이언트 TS는 화면 보조일 뿐이다.

어디	프로젝트	하는 일	접근
Nuxt 클라이언트 didLogic.ts	jpd-did	루트가 끊겼는지 + 공격영역 진입 여부 → 선수 입력 버튼 띄우기 전용	누구나 상관없음
KPI 서비스 python	jpd-did	DAP/DTP/STP/UTP 분류, B·M·A·S 역할, BAP, KPI 집계	개발자 포함
평점 서비스 python	jpd-rating 별도 프로젝트	선수평점·팀지수·등급, 기준값, 채번 규칙, dMST/playerMST 조립 및 JaionX 쓰기	1~2명

클라이언트에 남는 판정은 하나뿐이다 — "어느 레코드에 선수를 물어볼까". DTP/DAP라는 이름을 붙이는 일은 전부 파이썬 이 한다. 클라는 ① 루트가 끊겼나(4초 공백·결과) ② 공격영역에 들어갔나(`area < 7`) 두 가지만 보면 되고, 이는 분류 로직이 아니다.

클라 규칙이 다소 넉넉해도 문제없다. 버튼이 몇 개 더 떠서 선수가 더 붙을 뿐이고, 파이썬이 DAP가 아니라고 판정하면 집계에서 빠진다. 반대로 덜 뜨면 데이터가 비므로, 넉넉한 쪽이 안전하다.

2.1 클라이언트 판정 결과는 저장하지 않는다

`didLogic.ts` 가 만든 플래그(`isDap` , `isDts` ...)는 화면에 버튼을 띄우는 데만 쓰고 버린다. 저장되는 것은 사람이 찍은 사실뿐이고, 파이썬이 그것을 읽어 다시 판정한다. 두 구현이 어긋나도 JaionX에 틀린 값이 가지 않는다.

2.2 그런데 왜 클라이언트에 판정이 남아 있나

선수 입력 요청 횟수 때문이다. 실측(2026-08-21 ARS):

방식	경기당 선수 입력 요청	
DAP 판정 사용 (현재)	457회	<code>DAP</code>
엑트마다 전부 물어봄	861회	<code>gi_tmp</code> — 거의 2배. 실시간에서 못 따라감

DAP 판정이 곧 "선수를 물어볼 최소 집합"이다. 이것이 클라에 로직을 두는 유일한 이유다. 향후 비전이 등번호를 채우면 이 버튼 자체가 사라지고, 클라이언트 판정도 제거해 파이썬 단일 구조가 된다.

03 가장 중요한 결정 — matches ≠ recordings

레거시가 이미 이렇게 하고 있었다. 새로 고안한 것이 아니라 보존하는 것이다.

```
// lib/extra.lib.php:3799
$gi[$row['gi_write_code']] = $row;      // H/A 를 키로 담고
if (count($gi) < 2) return FALSE;      // 2개가 아니면 실패
// 주석: "양팀 입력 모두 종료되지 않은 경기는 진행 불가"
```

ff_game_info 에는 gi_write_code (H/A)와 gi_user_id (기록자)가 있고, KPI 집계는 전부 이 테이블에 있다. 즉 KPI는 경기 단위가 아니라 팀-기록자 단위다. 팀 판별도 전부 if(gi_write_code='H', gi_h_t_code, gi_a_t_code) 패턴이다.

그리고 이건 지금 앱과 일치한다 — TeamSelection의 "입력할 팀 선택"(game.team)이 곧 gi_write_code 다.

3.1 바뀌는 것은 표현 방법뿐

	레거시	새 스키마
관계	ff_game 1행 : ff_game_info 2행	matches 1문서 : recordings 2문서 (동일)
소속 표현	gm_id 컬럼 + JOIN	matches/{id}/recordings/{H A}
H/A 구분	gi_write_code 컬럼	문서 ID 자체
세션 키	gi_id (숫자)	불필요 — 경로가 키. legacyGiId 만 매핑용으로 남긴다
중복 세션	가능 (막을 장치 없음)	구조적으로 불가능

Firestore에는 JOIN이 없다. 소속을 표현하는 방법은 ① 필드에 부모를 적고 질의 ② 경로에 넣기 두 가지인데, ②는 색인이 필요 없고 질의가 아닌 **직접 읽기**이며 결과가 "있거나 없거나" 둘 중 하나다. matches/{matchId}/recordings/H — 이 주소 한 번 읽기로 복구가 끝난다.

04 전체 트리

컬렉션	경로	레거시	대응
-- 경기 기록 -----			
matches/{matchId}	← ff_game	일정·확정 스코어	
recordings/{H A}	← ff_game_info	★ 팀별 기록 세션	
records/{recordId}	← ff_game_record	플레이 1건	
subs/{subId}	← ff_game_player_log	교체	
cards/{cardId}	← ff_game_card	경고/퇴장	
paths/{pathId}	← ff_game_path	[확정 스냅샷]	
bapEvents/{bapId}	← ff_game_bap	[확정 스냅샷]	
playerStats/{playerId}	← ff_game_player	[확정 스냅샷]	
-- 참조 · 데이터 관리 -----			
leagues/{leagueId}	← ff_league		
seasons/{seasonId}	← ff_season		
stadiums/{stadiumId}	← ff_stadium		
teams/{teamId}	← ff_team		
seasons/{seasonId}	← (신규)	시즌별 리그 - 승강제	
squad/{playerId}	← (파생)	현재 스쿼드 뷰	
players/{playerId}	← ff_player		
contracts/{contractId}	← ff_player_team	★ 이적시장 원장	
recorders/{uid}	← ff_user		

만들지 않는 것

레거시	처리
ff_game_state	recordings.status 를 관리자가 고치는 것으로 대체
ff_session · ff_member_session	Firebase Auth 토큰이 대체
ff_formation	슬롯 좌표는 상수. TeamSelection.vue 에 두고 formationKey 만 저장

05 matches/{matchId} — 일정

두 기록 세션이 공유하는 사실만 담는다. 문서 ID는 gm_id 를 그대로 쓴다.

문서 ID는 gm_id를 그대로 쓴다

matches/20262027ep1j2rgn13na28rd2b8q0001

gm_id 는 레거시의 옛날 키가 아니라 **JaionX의 모든 테이블을 잇는 조인 키다.**
TACTIC · playerMST 의 첫 컬럼이 전부 gm_id 이므로, 신규 경기도 이것 없이는 JaionX에 넣을 수 없다.

20262027 ep1 j2rgn13na28rd2b8q 0001

8자 3자 17자 4자

시즌 리그 리그 고정 상수 일련번호

구간	내용	검증
[0:8]	시즌 20262027	
[8:11]	리그 ep1	
[11:28]	리그 고정 상수	2024-25 · 2025-26 · 2026-27 세 시즌 780건 전부 동일. 시즌별 난수가 아니다
[28:32]	일련번호	시즌 통산 0001~0380, 빠짐없이 연속. 라운드1 = 1~10, 라운드2 = 11~20

다른 리그를 추가할 때 상수가 달라지는지는 **확인 불가** — EPL 데이터밖에 없다. `gm_id` 를 문서 ID로 쓰면 매핑 필드가 필요 없고, export 시 **변환이 아예 없다**.

일정은 만들지 않고 가져온다

```
footballx/server/backend/data/matchschedule/{시즌}/match_schedule.json
— 임포트 —> matches/{gm_id}
```

```
{ "gm_id": "20262027ep1j2rgn13na28rd2b8q0020",
  "season": "2026-2027", "gm_date": "2026.08.31", "gm_round": 2,
  "home": { "code": "AVLX", "goal": 0, "jmx": 40.0, "points": 1, "sRound": 26270219 },
  "away": { "code": "ARSX", ... }, "is_pending": true, "status": "FORECAST" }
```

일련번호는 우리가 매기지 않는다

같은 시즌·같은 라운드에서 경기를 구분하는 것은 **일련번호 4자리**뿐이고, 그 번호는 일정이 만들어질 때 정해진다. 우리가 따로 발급하면 **같은 경기에 다른 번호가 붙어 JaionX와 영영 맞지 않는다.**

필드	타입	레거시	비고
(문서 ID)	string	gm_id	위 구조. 일정에서 그대로 가져온다
date / kickoffTime	string	gm_date, gm_time	
leagueId / seasonId / round	string / number	gm_league, -, gm_round	
stadiumId	string	gm_s_code	
homeTeamId / awayTeamId	string	gm_h_t_code / gm_a_t_code	일정의 home.code / away.code (IDX_CODE 체계)
sRound	{home, away}	-	일정에 이미 있는 D-MST S-Round 채번값
score	{home, away}	gi_goal_home/away	각 recording이 자기 쪽 필드만 merge (§5.1)
createdAt / updatedAt	Timestamp	-	

is_view · is_onair · is_old 는 옮기지 않는다. is_realtime 은 경기 속성이 아니라 기록자의 입력 모드이므로 recordings 로 내린다.

5.1 스코어는 누가 쓰는가

골은 각 기록자가 자기 팀 것만 찍는다. Firestore는 필드 단위 merge가 되므로 동시에 써도 충돌하지 않는다.

```
setDoc(matchRef, { score: { home: goals } }, { merge: true })
```

현재 코드 버그

DidInput.vue:455 의 recordGoalResult 가 기록 중인 팀과 무관하게 항상 homeScore.value++ 한다. 원정팀을 기록 중이어도 홈 스코어가 오른다. 연동 시 recordings.side 기준으로 고칠 것.

06 recordings/{H|A} — 팀별 기록 세션 ★

필드	타입	레거시	비고
side	'H' 'A'	gi_write_code	문서 ID와 중복이나 collectionGroup 질의를으로 유지
teamId / opponentTeamId	string	gi_h_t_code / gi_a_t_code	
recorderId	string	gi_user_id	Auth UID. 보안 규칙의 기준
status	ready H1 H1_done H2 H2_done final	gi_state	이미 코드에 있는 값 그대로. 화면 상태와 1:1이라 매핑 코드 불필요
inputMode	'분석' '실시간'	is_realtime	집계·전송 트리거를 가른다 (§11)
fieldSide	'left' 'right'	gi_part	전반 기준 진영. 구역코드 반전
formationKey	string	gi_formation	
lineup	map	ff_game_player	§6.2
halves	map	gi_h1_begin, gi_h1_seconds ...	§6.1
maxSeq	number	—	NEW 레코드 정렬키 발급 (§7.2)
kpi	map null	gi_tap, gi_bap ...	SNAPSHOT §08
kpiComputedAt	Timestamp null	—	신선도 판단
legacyGiId	number null	gi_id	NEW JaionX 동기화 매핑용. 신규 경기는 null
syncedAt	Timestamp null	—	동기화 역등성
createdAt / updatedAt	Timestamp	gi_regdt / gi_moddt	

6.1 halves — 시간의 기준을 여기서 잡는다

```
halves: {  
  H1: { startedAt: Timestamp, seconds: 2712 }, // 종료 시 확정  
  H2: { startedAt: Timestamp, seconds: 2840 },  
}
```

레코드에는 half + halfSeconds 만 저장하고, 경기 전체 기준 초(gr_seconds)는
export 시점에 계산한다. 저장 안 함 = 어긋날 일 없음.

현재 코드 버그를 이 설계가 고친다

`DidInput.vue:35` 는 `seconds = ref(0)` 이고 후반에도 0부터 다시 센다. 그런데 `DidInput.vue:67` 에서 후반 진입 시 전반 레코드를 그대로 이어받으므로, 배열 안에 **H1의 30초와 H2의 30초**가 같은 값으로 공존한다.

`resolvedOnly()` 가 `seconds` 로 정렬하는 순간 전·후반 레코드가 뒤섞이고 4초 롤이 반 경계를 넘는다.

그리고 이걸 **JAIONX** 요구사항이기도 하다

`2627-5분단위 통합.xlsx` 는 전·후반 × 5분 구간 20칸(`JMX-H1-0-5 ... DAP-H2-45-Plus`)으로 되어 있고, 팀 **JMX**가 이 시계열에서 산출된다(§9.3). 반 구분과 반 기준 경과초가 없으면 이 시트를 만들 수 없다.

6.2 lineup — 서브컬렉션이 아니라 map

18명을 항상 통째로 읽고 쓴다. 서브컬렉션이면 로드마다 18 read, map이면 1 read다.

```
lineup: {
  [playerId]: {
    slot: 'o0' | 'gk' | 'b3',    // TeamSelection 의 assigned 키
    order, type: 'START' | 'BENCH',
    no: '7', name: 'B. Saka', pos: 'FW',    // ★ 스냅샷
    inHalf, inSeconds, outHalf, outSeconds,
  }
}
```

이름·번호를 복사하는 이유(원칙 07): 선수가 이적하거나 번호를 바꾸면 `players` 문서가 갱신된다. 스냅샷이 없으면 작년 경기를 열었을 때 그때 뛰지도 않은 팀의 번호로 표시된다.

선수별 지표(`gp_tap` , `gp_score` ...)는 여기 두지 않는다 — 파생값이고 목록 화면을 무겁게 한다. 확정 시 `playerStats` 로 뺀다.

07 records/{recordId} — 플레이 1건

`didLogic.ts` 의 `DidRecord` 가 곧 문서 스키마다. 새 타입을 만들지 않는다.

필드	레거시	비고
(문서 ID)	gr_id	Firestore 자동 ID (§7.2)
half	gr_half	'H1' 'H2' 'H3' 'H4'. 반드시 저장 (§6.1)
halfSeconds	gr_half_seconds	해당 반 기준 경과초. 4초 물의 입력값
seq	gr_regdt 대응	같은 초 안의 순서 (§7.2)
act / res	gr_act_code / gr_res_code	GB / GX / GOAL 등 신 코드 체계
area	gr_area_code	1~18 구역
posX / posY	gr_pos_x / gr_pos_y	0~100%. 레거시는 픽셀값이다 — 변환 필요 (§15)
shootPosX / shootPosY	gr_shoot_pos_x/y	골대 타깃 좌표
shootDspRange	—	NEW 골포스트 1m 판정 캐시
isShot	—	NEW write-back으로 act가 비워져도 원래 쏘았음을 보존
playerId	p_id	'OWN' = 자책골 (레거시 실데이터에서 확인, §13)
source	—	NEW 'did' 'vision'. 지금은 전부 'did'
playerIdSource	—	NEW 선수를 누가 넣었나
createdAt	gr_regdt	

SOURCE 분기를 지금 넣어두는 이유

비전 데이터 구조는 지금 설계하지 않는다 (§15). 다만 필드가 없으면, 나중에 비전이 들어왔을 때 과거 레코드에 대해 "이 값은 사람이 넣은 건가 기계가 넣은 건가"를 영원히 알 수 없다. 마이그레이션을 피하려고 자리만 잡아둔다.

원칙은 하나다 — 비전은 DID 값을 덮어쓰지 않고 옆에 붙는다. 되돌릴 수 있어야 하고, 정확도를 재려면 사람 입력과 대조할 수 있어야 한다.

7.1 저장하지 않는 것 — gt_id 와 gr_is_* 14개

레거시는 레코드마다 소속 루트와 판정 플래그 14개를 저장했다. 여기서는 전부 뺀다.

레거시가 저장한 이유	지금은?
계산 로직이 서버(PHP)에만 있었다. 여러 클라이언트가 각자 계산할 수 없었다	사라짐. didLogic.ts 를 모든 화면이 공유
SQL 집계가 필요했다 — SUM(gr_is_tap) GROUP BY 는 컬럼이어야 가능	사라짐. 확정 스냅샷을 따로 저장 (§08)
2013년 단말에서 1,200건 재계산이 부담이었다	사라짐. 실측 0.16ms (§13)

그리고 결정적으로, 레코드 하나를 수정하면 앞뒤 루트의 플래그가 연쇄로 바뀐다. 시간 1건을 고치면 4건 × 14개 = 56개 값이 바뀐다. 수정 기능이 이미 있으므로 (DidInput.vue:157) 저장하면 반드시 어긋난다.

단, GT_ID는 JAIONX가 요구한다

TACTIC.xlsx 에 gt_id 컬럼이 있다. 저장하지 않되 export 시점에 computeAttackPaths() 의 path.gtId 를 채워 내보낸다. 확정 스냅샷의 paths (§08)가 그 자리다.

7.2 정렬키 — 지금 코드의 ID 충돌을 막는다

현재 코드 버그

didLogic.ts:121 의 let recSeq = 0 은 모듈 전역 카운터다. 새로고침하면 0으로 돌아가므로, 저장된 기록을 이어서 입력하면 rec_1 이 두 번 생기고 seq 도 중복된다.

문서 ID는 Firestore 자동 ID를 쓰고, seq 는 세션 로드 시 recordings.maxSeq 에서 이어받는다. 정렬은

orderBy('half').orderBy('halfSeconds').orderBy('seq') — seq

단독 정렬은 불가하다. 수정으로 시간을 바꾸면 입력 순서와 시간 순서가 어긋나기 때문이다.

7.3 자책골 처리 — playerId = 'OWN'

레거시 실데이터에 p_id = 'OWN' 이 그대로 들어 있다. 집계 규칙은 D-MST와 대조해 확정했다:

GOAL = 자책골 포함
SHOT = 자책골 제외
SSR = (GOAL - OG) / SHOT
OG = 자책골 수 (별도 컬럼)

08 파생 데이터는 언제 저장하나

시점	동작
기록 중 (H1~H2_done)	저장하지 않는다. 화면은 records로 즉석 계산
확정 (status: 'final')	paths / bapEvents / playerStats / kpi 를 한 번에 배치 write
확정 후 수정	스냅샷 무효화 → 재계산 → 덮어쓰기 (kpiComputedAt 갱신)

변동기와 확정기의 요구가 반대이기 때문이다. 기록 중에는 매 탭마다 값이 바뀌므로 저장에 곧 부채고, 확정 후에는 리포트·동기화가 "그때 값"을 안정적으로 요구한다.

`AttackPath.recordIds`가 레코드를 참조하므로, 삭제 가능한 기록 중에 저장하면 끊어진 참조가 남는 것도 이유다.

이전 판의 오류를 정정한다

스냅샷 필드 목록의 기준은 **화면이 아니라 JaionX**다. 이전 판에서 "화면에 보이는 8개 중심"이라고 쓴 것은 잘못이었다. §09의 컬럼 전체를 담는다.

09 출력 규격 — JaionX가 받는 것

이것이 규칙 0의 실체다. 규격의 원본은 footballx repo에 있다.

대상	규격 위치	성격
dMST	admin/dMST.vue tableHeaders	팀-경기 단위. "전 시스템의 중심 허브"
playerMST	admin/playerMST.vue	선수-경기 단위. 레거시 <code>gp_*</code> 거의 그대로
multiSheet	admin/multiSheet/index.vue	헤더 고정 아님. 업로드 행의 키에서 동적 생성
TACTIC	레코드 원본	gm_id, p_id, t_code, gr_half, gr_half_seconds, gr_area_code, gr_act_code, gr_res_code, gr_pos_x/y, gt_id ...

9.1 이미 Firestore다 — CSV는 껍데기다

```
// footballx/app/composables/dMST.ts
import { addDoc, collection, ... } from 'firebase/firestore'
const q = query(collection(db, 'dMST'), ...)
```

"CSV를 admin에 올린다"는 결국 Firestore에 문서를 쓰는 것이다. 우리는 CSV를 만들 필요가 없다. 단 프로젝트가 둘이다 — didNew는 `jpd-did`, JaionX는 `aifootballx`. 클라이언트끼리는 서로 쓸 수 없으므로 **반드시 서버(파이썬)를 거친다**. 이는 대외비 격리와의 맞아떨어진다.

9.2 dMST 컬럼 중 우리 몫

구분	컬럼	담당
DID 산출 KPI	gi_tmp(=TAP), DAP, TTP, DTP, BAP, B, M, A, S, SHOT, ASR, SSR, GOAL, OG	우리
경기 메타	gm_round, gm_date, t_code, t_Nm, H-A	우리
채번/식별	Season, S-Round, M-Code, Match, T-Round, IDX_CODE, PlSeason	규칙 파악됨 → 대외비 서버
AI 예측	X-*, *(Ai*)	파이썬 분석 쪽
스코어링	JMX, APX, TPX, FPX, *Score, Power3X, G-*, W-TOT, Point, PDF	대외비 서버

B/M/A/S 는 빌더·메이커·어시스터·슈터 수 = gr_is_ctb/ctm/cta/cts 집계다.
저장하지 않아도 computeAttackPaths() 로 산출된다는 것을 실데이터로 검증했다 (§13). H-A 는 gi_write_code 그대로 = 우리 recordings 문서 ID .

9.3 팀 JMX는 5분 구간 시계열에서 나온다

경기 전체 집계가 아니라 전·후반 5분 구간 20칸의 누적값을 정규화한 뒤 평균내어 산출한다.
그래서 §6.1의 half / halfSeconds 가 필수다.

9.4 대외비 — KPI가 쌓이는 곳과 평점이 쌓이는 곳을 분리한다

선수평점(P-JMX , P-APX/TPX/FPX , 등급)과 팀지수(JMX , Power3X)의 산출식·기준값·가중치·채번 규칙은 대외비다. 이 repo와 클라이언트 번들에 넣지 않는다.



#	규칙
01	평점 서비스만 KPI를 읽는다. 반대 방향은 없다. KPI 쪽에서는 평점이 존재하는지도 모른다
02	aifootballx 쓰기 권한도 평점 서비스만 가진다. dMST 한 행에 KPI 컬럼과 평점 컬럼이 섞여 있어 어차피 마지막에 합쳐야 하므로, 합치는 지점을 평점 쪽에 두면 서비스 계정이 하나로 줄어든다
03	기준값도 평점 저장소에만 둔다. KPI 프로젝트를 전부 뒤져도 <code>DAP 65, BAP 7, SHOT 3</code> 같은 숫자만 있고 점수는 없다

접근 통제는 두 층이다 — **실행 접근**(누가 호출하나)과 **소스 접근**(누가 코드를 읽나). 후자가 본질이며, 배포된 소스는 프로젝트 IAM 권한자가 내려받을 수 있으므로 **프로젝트 분리**가 가장 확실하다.

무엇이 지켜지고 무엇이 안 지켜지나 — 솔직한 평가

막을 수 없는 것

JaionX는 선수별 **입력(P-DAP, P-BAP ...)**과 **출력(P-JMX, 등급)**을 같은 표에 함께 보여준다. 따라서 **작성하고 회귀분석을 하면 공식은 복원된다**. 6개 입력의 선형 조합 + 상한 클램프라 어렵지도 않다.

그러므로 **API 호출 제한·역산 방지 장치는 만들지 않는다**. 이미 화면에 있는 것을 API로 막는 셈이라 실익 없이 복잡도만 늘린다.

지켜지는 것	내용
복사 경로 차단	권한 없는 개발자에게 "소스를 그대로 가져가는" 길이 없다. 남는 건 회귀분석뿐이고 그건 별개의 일이다
정확도와 시간	회귀는 근사치다. <code>0.99</code> 클램프, <code>99.9</code> 상한, X-GOAL의 계단식(0→50, 1→75, 2+→99.9), 등급 경계 같은 예외 처리는 잘 나오지 않는다
비공개 영역	채번 규칙과 중간값(KPI-ALL, Conversion, 기본점수, 골 보너스)은 화면에 없다
영업비밀 지위	비밀로 관리한 사실이 있어야 비밀이다

가장 강한 방어는 기술이 아니다

기준값을 사람이 조정하는 것 자체가 가장 강한 방어다. 회귀로 과거 값은 맞출 수 있어도, 다음에 어떤 값으로 바꿀지는 규칙이 아니라 판단이라 예측할 수 없다.

반대로 **"기준값 = 시즌 평균"**으로 자동화하면 그것도 규칙이 되어 복원 가능해진다 — 평균을 낼 재료가 JaionX에 이미 공개돼 있기 때문이다. 자동화는 보안이 아니라 **운영 편의**를 근거로 판단할 것(§15).

ratingBaseline — 기준값은 반드시 버전으로 저장한다

```
ratingBaseline/{version} → { player: {DAP:30, BAP:8, DTP:6, SHOT:6, SSR:0.33, GOAL:2},
                                team:   {DAP:400, BAP:60, DTP:12, SHOT:15, SSR:0.18, GOAL:3},
                                weights: {...}, effectiveFrom, createdBy }

ratings/{...}/{playerId} → { pJmx: 11.65, ..., baselineVersion: 'v3' }
```

기준값은 데이터를 보며 자주 바뀐다. 어떤 기준값으로 계산된 평점인지 같이 남기지 않으면, 기준값을 바꾸는 순간 과거 평점이 전부 흔들리고 왜 달라졌는지 설명할 수 없다. 지금은 시즌별 엑셀 파일이 따로 있어 우연히 보존되고 있으나, 자동화하면 그 우연이 사라진다.

9.5 playerStats가 채우는 것

playerMST 의 gp_* 컬럼이 그대로 목록이다 — gp_tmp, gp_tap, gp_utp, gp_ctp, gp_ttp, gp_sht, gp_ast, gp_goal, gp_ctb, gp_ctm, gp_cta, gp_cts, gp_gtb, gp_gtm, gp_asr, gp_ssr. 단 gp_score_rel / gp_score_abs / gp_score (평점) 3개는 대외비 서버만 채운다.

10 데이터 관리 — 이적시장 · 팀 정보

경기 기록이 아니라 관리 화면이 쓰는 참조 데이터. 입력·수정 대상이므로 이력과 감사 필드가 필요하다.

10.1 players — 정체성만

name/nameEn/nameFull, birth/height/foot/nation, active (은퇴 대신 비활성 — 과거 기록을 고아로 만들지 않는다), currentTeamId/currentNo/currentPos (파생 캐시), legacyPlayerId. 어느 팀 소속인지는 시간에 따라 변하므로 여기 두지 않는다.

10.2 players/{id}/contracts — 이적시장 원장 ★

레거시 ff_player_team 이 이미 시간축 테이블이다. 이를 추가 전용 원장으로 만든다.

필드	레거시	비고
teamId / leagueId / seasonId	t_code / l_code	
from / to	pt_begin / pt_end	to = null 이면 현재 소속
no / pos	pt_num / pt_pos	계약에 종속 (팀 옮기면 바뀐다)
fromTeamId	-	NEW 직전 소속팀 → 한 문서로 "A→B" 완성
transferType	-	NEW TRANSFER / LOAN / LOAN_RETURN / FREE / YOUTH / RETIRE
fee / currency	-	NEW 선택

이적 한 건 = 트랜잭션 하나 — ① 이전 계약 to 닫기 ② 새 계약 생성 ③

players.current* 갱신 ④ 양 팀 squad 갱신. 부분 실패하면 스쿼드가 어긋난다.

이적 목록은 collectionGroup('contracts').orderBy('from', 'desc')

한 방이다 — 계약 문서가 곧 이적 이벤트다.

10.3 teams/{id}/squad — 현재 스쿼드 파생 뷰

원장만 있으면 계약 30건 + 선수 30명 = **61 read + 조인**이 필요하다. 이 뷰가 있으면 **1 query**로 끝난다. 갱신은 이적 시에만(연 수십 회)이라 어긋날 위험이 낮다 — 원칙 03의 정당한 예외. 현재 app/utils/squads.ts 의 하드코딩이 이 자리를 임시로 메우고 있다.

10.4 teams — 승강제 대응

l_code 를 팀 문서에 단일값으로 두면 승강제를 표현할 수 없다. 강등되면 작년 리그가 덮어써진다.

```
teams/{teamId}/seasons/{seasonId} → { leagueId, division, finalRank? }
```

currentLeagueId 는 목록 필터용 캐시로만 유지한다.

10.5 참조 컬렉션 · 감사 필드

leagues, seasons, stadiums, recorders. u_pw 는 옮기지 않는다 — 비밀번호는 Auth가 관리한다. 관리 대상 문서 전부 createdAt / createdBy / updatedAt / updatedBy 를 붙인다. 경기 기록에는 붙이지 않는다 — 세션이 이미 recorderId 로 귀속된다.

11 쓰기·조회 전략

11.1 레코드는 두 모드 모두 즉시 저장한다

방식	판단
메모리에 모았다가 반 종료 시 flush	❌ 지금 겪는 문제 그대로. 새로고침 한 번에 반 전체가 날아간다
탭할 때마다 즉시 write	✅ 채택 (두 모드 공통)

핵심은 Firestore의 **오프라인 지속성**이다. `persistentLocalCache` 를 켜면 쓰기가 먼저 IndexedDB에 떨어지고 UI는 즉시 반영되며, 네트워크가 돌아올 때 자동 동기화된다.
경기장 와이파이가 끊겨도 입력이 멈추지 않는다.

11.2 모드가 가르는 것은 집계·전송 트리거다

무엇	경기당 비용
레코드 1,200건 Firestore 쓰기	약 3원
파이썬 집계 실행 + JaionX 갱신	실행 횟수에 비례 — 여기가 실제 비용

분석	레코드 → Firestore 즉시 저장 ↳ 집계·JaionX 전송은 [전송/갱신] 누를 때만
실시간	레코드 → Firestore 즉시 저장 ↳ 집계·JaionX 전송도 계속 (5~10초 묶음 권장)

`useMatchState` 는 버리지 않고 **Firestore 구독의 로컬 캐시**로 역할만 바꾼다.

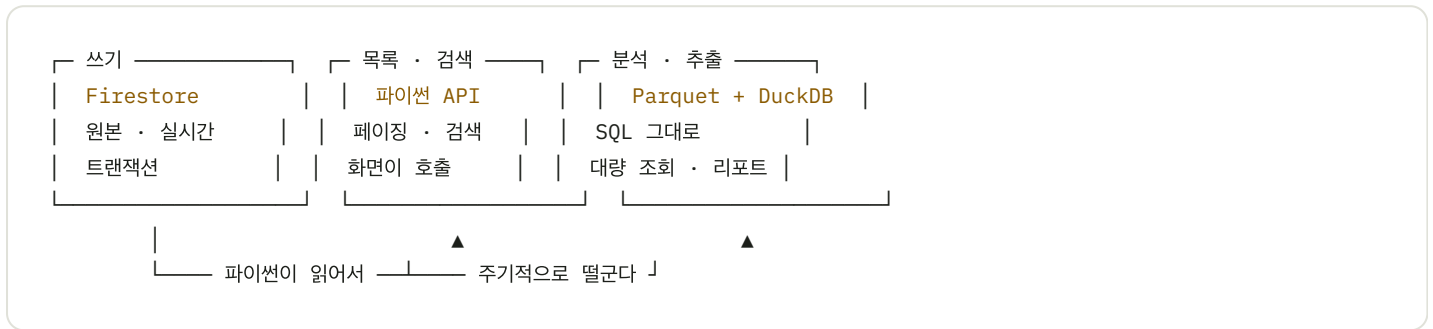
11.3 조회 층 — 화면은 Firestore를 직접 나열하지 않는다 ★

규칙
목록·검색·집계를 <code>getDocs</code> 로 컬렉션을 훑어서 해결하지 않는다. 이 규칙이 없으면 화면을 만들 때마다 전량 로드로 되돌아간다.

지금 footballX가 겪는 문제가 그것이다.
`footballx/app/composables/dMST.ts` 는 조건에 맞는 문서를 **전부 받아서 배열에 쌓는다.**

컬렉션	시즌당 문서	한 번 열 때
dMST	380경기 × 2팀 = 760	760 read
playerMST	11,500	11,500 read + 수 MB 전송

Firestore의 구조적 한계 때문에 이렇게 될 수밖에 없다 — 필드를 골라 받을 수 없고(웹 SDK에 `SELECT col1, col2` 가 없다), **검색이 없고**(`LIKE '%손흥민%'` 불가, 앞글자 일치만 전부라 결국 **전부 받아서 브라우저에서 필터** 하게 된다), **조인·집계가 없다**.



	FIRESTORE	파이썬 API	PARQUET + DUCKDB
실시간 쓰기 · 오프라인	✓		
검색 · 정렬 · 페이징	✗	✓	✓
집계 · 조인 · SQL	✗		✓

분석 층은 새로 만드는 것이 아니다.

`footballx/server/backend/data/**/*.parquet` 가 이미 그 구조다.
DuckDB만 얹으면 SQL이 그대로 된다.

```
SELECT t_code, AVG(DAP), SUM(GOAL)
FROM 'dMST_*.parquet'
WHERE gm_round BETWEEN 1 AND 10
GROUP BY t_code ORDER BY 2 DESC;
```

11,500행이 밀리초 단위로 처리되고, **Firestore read 비용이 0**이다.

화면이 Firestore를 직접 읽어도 되는 경우

- 문서 하나를 주소로 읽을 때 — `matches/{gm_id}/recordings/H` 같은 직접 읽기
- 실시간 구독이 필요한 기록 화면 — DID 입력 중의 `records`
- 방금 자기가 쓴 문서를 되읽을 때 — 스냅샷 갱신 전에 화면에 반영하기 위해

대상	색인
레코드 정렬	records: half ASC, halfSeconds ASC, seq ASC
기록자 대시보드	CG recordings: recorderId ASC, createdAt DESC
일정 목록	matches: date ASC, leagueId ASC
이적 목록	CG contracts: from DESC
팀별 계약	CG contracts: teamId ASC, to ASC

13 실데이터 검증 ★

2026-09-02. 레거시 `ff_game_record` 실제 1,536행(라운드 2, 20개 팀-경기)을 입력으로 `didLogic.ts`를 수정 없이 실행하고 D-MST 값과 대조했다.

항목	결과
DTP 슛 포함 공격루트	20 / 20 팀 일치
GOAL	20 / 20 일치
SHOT	자책골 제외 후 20 / 20 일치
B / M / A / S	17 / 20 일치 (3팀에서 1~2 차이)
공격루트 그룹핑	382개 전부 레거시 <code>gt_id</code> 경계 안. 잘못 합친 것 0건

우리 4초 룰은 레거시 273루트를 382개로 더 잘게 쪼갬다. 쪼개진 조각은 슛이 없어 UPP(무효)가 되므로 DTP 개수에는 영향이 없었다.

13.1 성능 실측 – 원칙 03의 근거

레코드	COMPUTEATTACKPATHS + COMPUTEBAP 전체 재계산 1회
300건	0.11 ms
600건	0.15 ms
1,200건 한 경기	0.16 ms

한 프레임(16ms) 안에 100번 돌려도 남는다. 계산이 저장보다 압도적으로 싸다.

13.2 발견한 버그 — 자책골이 SHOT에 포함된다

```
// didLogic.ts:415 - 공통 플래그 루프
for (const r of chain) {
  const { act, res } = r          // ← 원본 act. isOwnGoal 보장 없음
  f.isTap = !!act
  f.isSht = isShotAct(act) || !!r.isShot
```

1차 패스(:333)와 2차 패스(:380)는 `const act = isOwnGoal ? '' :`
`r.act` 로 보장하는데 이 루프만 빠졌다. 파일 상단 주석은 "자살골은 TAP, DAP, DTP, Shoot
집계에서 제외"라고 명시하고 있다. 자책골을 제외하고 다시 돌리니 SHOT이 3팀 불일치 →
20/20 일치로 바뀌었다.

13.3 남은 불일치

팀	차이	추정
BHAX	S 15/14	H2 2926s H/O — 결과가 안 찍힌 슈트. 우리는 closeTrailing으로 마감해 슈터를 부여, 레거시는 미부여
LIVX	A 11/10, S 13/12	미확정
TOTX	B 19/17, M 6/5, A 11/10, S 16/15	루트 하나(roles=BBMAS)가 통째로 빠진 것과 정확히 일치. 이유 미확정

규명하려면 `gr_is_cts/cta/ctm/ctb` 가 포함된 export가 필요하다. 현재 파일에는
없어 총합만 비교 가능하다.

13.4 아직 검증되지 않은 것

주의

TACTIC.xlsx에는 슈트가 포함된 루트만 들어 있다(273루트 전부 슈트 존재, 슈트 없는 루트 0개). 따라서 `gi_tmp` (TAP), `DAP`, `TTP`, `BAP`, `ASR`는 한 번도 실데이터로 검증된 적이 없다. 전체 레코드가 포함된 export가 필요하다.

14 레거시 대비 — 무엇이 사라지고 무엇이 나아지나

사라지는 것

레거시	왜
gi_id (세션 키)	경로가 키를 대신한다. 매핑용 <code>legacyGiId</code> 만 남긴다
레코드마다의 gi_id (1,200회 중복)	서브컬렉션 경로가 소속을 표현한다
gr_is_* 14개 + gt_id	파생값. 0.16ms에 재계산된다
dplay_game_path_reset 배치	저장을 안 하므로 재계산 대상이 없다
ff_game_state	<code>recordings.status</code> 수정으로 대체
ff_session / ff_member_session / u_pw	Firebase Auth
ff_formation	상수
PHP + MySQL 서버 운영	관리형 서비스
엑셀 수작업 6단계	파이썬 자동 변환 (§00)

없어지는 엑셀 편법

지금	무슨 문제	새 구조
IF(MOD(Match,2)=1, 다음 행, 이전 행)	상대팀을 인접 행으로 찾는다. 정렬이 깨지면 승점이 조용히 틀어짐	matches의 반대편 recordings를 읽는다
IF(윗 행이 비었으면 "H" else "A")	행 위치로 홈/원정 판정	문서 ID가 곧 H/A
Point = W-TOT2 - W-TOT	베이스 숫자를 더했다 빼는 우회	골 비교로 직접 산출
'C:\EPL2425\[...24R.xlsx]P-KEY'!J4	라운드마다 사람이 셀 링크를 손으로 수정	불필요

개선되는 것

	지금 (PHP / SQL)	새 구조
탭당 네트워크	HTTP 왕복 1회 + 응답 대기	0회 (로컬 우선, 백그라운드 동기화)
탭당 SQL	2~4개	—
오프라인	불가 (입력 중단)	가능
새로고침	—	살아남음
파생값 정합성	저장 + 배치로 맞춤	저장 안 함 = 어긋날 자리 없음
JaionX 반영	사람이 6단계	자동
대외비 로직	엑셀 파일로 유통	서버에 격리, 접근자 1~2명

문제였던 것 – 이 스키마가 고치는 현재 코드의 결함

01 새로고침하면 기록 유실

- 즉시
저장 +
오프라인
지속성
(§11)

02 후반 seconds가 0부터 재시작 → 전·후반 레코드가 섞이고 4초 룰이 반 경계를 넘음

- half
명시
저장
(§6.1)

03 recSeq 모듈 카운터 → 새로고침 후 ID·seq 중복

- 자동 ID
+
maxSeq
시드
(§7.2)

04 원정팀 기록 중에도 homeScore++

- side
기준
merge
(§5.1)

05 파생 플래그 저장 시 수정마다 어긋남

- 미저장,
확정 시
스냅샷
(§7.1,
§08)

06 이적하면 과거 경기의 선수 이름·번호가 바뀜

- 라인업
스냅샷
(§6.2)

07 스쿼드가 squads.ts에 하드코딩

→ 계약
원장 +
스쿼드
뷰
(§10.2)

08 자책골이 SHOT에 포함됨 — 실데이터로 발견

→ didLogic.ts:415
보정 (§13.2)

15 아직 결정하지 않은 것

- **기준값을 고정할 것인가, 시즌 평균으로 자동 갱신할 것인가** — 고정하면 과거 평점이 안 변해 **재현** 가능하다. 자동 갱신하면 리그 수준을 따라가지만 **어제 11.65였던 평점이 오늘 11.4가 된다**. 지금 엑셀은 사실상 고정. 어느 쪽이든 `ratingBaseline` 버전 스탬프가 있으면 과거 복원은 가능 (§9.4)
- **채번 규칙 구현 위치** — Season / S-Round / M-Code / Match / T-Round / IDX_CODE. 규칙은 파악됐으나 jpd-rating에 돌지 일반 파이썬에 돌지 미정
- **좌표 변환** — 레거시 `gr_pos_x/y` 는 픽셀(예: 327, 598), 우리는 0~100%. 기준 해상도 확인 필요
- **D-MST 실제 업로드 열** — 계산 시트는 221열, admin 화면은 60여 열. 어디까지가 업로드 대상인지
- **BAP가 팀 지수에 안 쓰이는 것이 의도인지** — `BAP(AiBuildup)` 열에 수식이 없고 JMX 평균에서도 빠져 있다. 기준값만 남아 있음
- **비전 데이터 구조** — 별도 컬렉션 형태, 좌표 형식, 타임스탬프 정합. **지금은 source / playerIdSource** 분기만 남기고 설계는 보류
- **연장전(H3/H4)** — 스키마는 열어뒀으나 UI는 전·후반만
- **확정 후 수정 시 재계산 트리거** — 클라이언트 즉시 재계산 vs 파이썬 위임
- **gp_score_rel / gp_score_abs** — 대외비 평점 외 나머지 두 지표의 산출 근거

레거시 전체 필드 · Aimbroad_database_schema-240812의 전체주요목록 시트

판정 로직 규칙 · docs/03_kpi_terminology.md + app/utils/didLogic.ts 주석 — 이 문서는 저장 스키마만 다룬다

출력 규격 · footballx/app/pages/admin/{dMST,playerMST,multiSheet} + composables/*.ts

대외비 산출식 · repo 밖 별도 관리 (§9.4)